

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Industry Problem Solving Task (Medium)

Assessment Tool Weightage: 35%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	5
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	5
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	5
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	5
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	5
6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	5
7	For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.	BL4 – Analyze	5

8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	5
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	5
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	5

Code of Conduct:

*I **Kadhirezhilan. D [192511228]** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1. E-Commerce Company – 10 Million Product Records

Question

An e-commerce company needs to store **10 million product records**. Recommend a file organization strategy and justify your choice with cost analysis.

Answer

1. Problem Understanding

An e-commerce platform with **10 million product records** represents a large-scale database. The system may perform different operations such as:

- Adding new products.
- Updating product information.
- Searching products using ProductID.
- Retrieving products by category.
- Filtering products by price.
- Displaying products for customers.
- Performing administrative updates.

Therefore, the file organization should provide a good balance between **storage efficiency, search performance, insertion performance, and scalability**.

For a large product catalog, continuously scanning all 10 million records for every search would be highly inefficient.

2. Recommended Strategy

I recommend a **Sorted File Organization combined with a B+ Tree primary index**. The product records can be physically stored according to a stable key such as:

ProductID

and a B+ Tree can be created on ProductID.

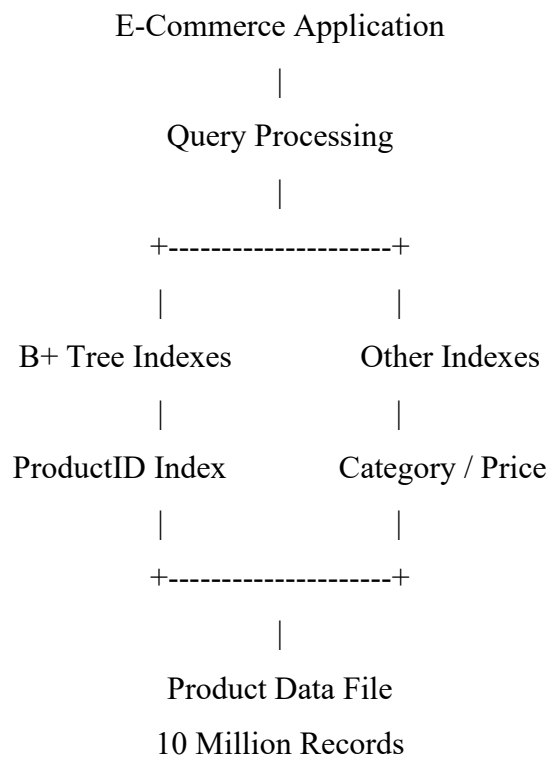
Additional secondary indexes can be created for frequently used attributes such as:

CategoryID

BrandID

Price

Proposed architecture



3. Why B+ Tree?

A B+ Tree is highly suitable for database systems because:

1. It keeps keys ordered.
2. It supports efficient point searches.
3. It supports range queries.
4. It remains balanced after insertions and deletions.
5. Its height remains small even for very large datasets.
6. Leaf nodes can be linked for sequential range retrieval.

For example, a price query:

Find products where price is between ₹500 and ₹1,000

can efficiently use an ordered B+ Tree index.

4. Cost Analysis

Assume each product record occupies approximately **500 bytes**.

Total raw data:

$$\begin{aligned} & 10,000,000 \times 500 \\ & = 5,000,000,000 \text{ bytes} \end{aligned}$$

Approximately:

4.66 GB

before considering indexes, metadata, replication, and storage overhead.

If the system used a sequential heap file, searching for one product could require examining a large portion of the 10 million records.

With a B+ Tree, the number of disk accesses is approximately proportional to:

4.66 GB

where:

- N = number of records
- B = number of keys per B+ Tree node.

Because a database block can contain many keys, the tree height remains relatively small.

5. Performance Comparison

Operation	Heap File	Sorted + B+ Tree
Insert	Very fast	$O(\log N)$
ProductID Search	$O(N)$	$O(\log_B N)$
Range Search	Poor	Excellent
Sequential Scan	Good	Good
Ordered Retrieval	Poor	Excellent
Scalability	Moderate	Excellent

6. Industry Justification

For an e-commerce system, **search and retrieval are more frequent than bulk insertion**. Therefore, slightly more expensive insert/update operations are acceptable in exchange for much faster product retrieval.

A pure heap organization would be attractive for insertion but inefficient for millions of searches.

A pure sorted file would provide good ordered retrieval but maintaining physical ordering during frequent product insertions can be expensive.

Therefore, a **B+ Tree indexed file organization** provides the best overall balance.

Conclusion

For 10 million e-commerce products, a **sorted/organized product file with B+ Tree indexing on ProductID and secondary indexes on frequently searched attributes** is the recommended solution. It provides scalable retrieval, efficient range queries, balanced storage access, and good performance for a large product catalog.

2. Banking System – Account Number and Date Range

Question

A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.

Answer

1. Problem Understanding

A banking transaction table may contain millions or billions of records.

A typical transaction record can be represented as:

TransactionID
AccountNumber
TransactionDate
Amount
TransactionType

Balance

The system must support queries such as:

```
mysql> SELECT *
-> FROM Transactions
-> WHERE AccountNumber = 10025;
```

and:

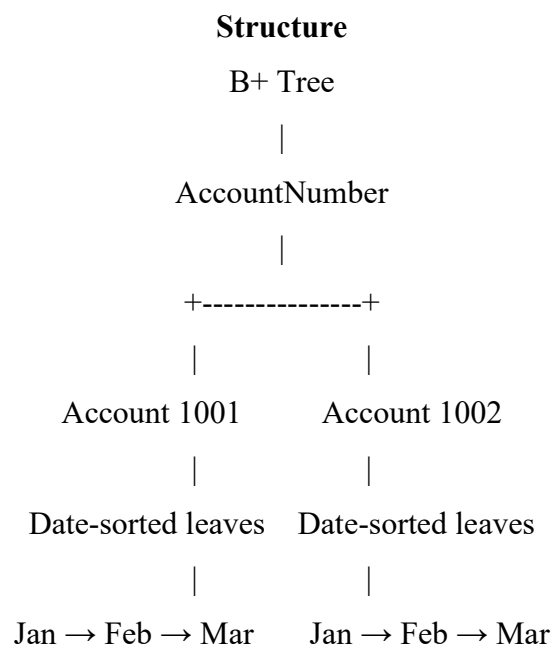
```
mysql> SELECT *
-> FROM Transactions
-> WHERE AccountNumber = 10025
-> AND TransactionDate BETWEEN '2026-01-01' AND '2026-03-31';
```

The second query requires both **equality filtering on AccountNumber** and **range filtering on TransactionDate**.

2. Recommended Index

I recommend a **composite B+ Tree index**:

(AccountNumber, TransactionDate)



3. Why Composite B+ Tree?

The first column should be:

AccountNumber

because account number is generally used for equality matching.

The second column should be:

TransactionDate

because the date condition is a range query.

For example:

```
mysql> WHERE AccountNumber = 10025  
-> AND TransactionDate BETWEEN '2026-01-01' AND '2026-03-31'
```

The index can first locate the account and then efficiently scan the relevant date range.

4. Additional Index

A separate B+ Tree index on:

TransactionDate

can be useful if the banking system frequently executes queries across **all accounts** for a date range.

For example:

```
mysql> SELECT SUM(Amount)  
-> FROM Transactions  
-> WHERE TransactionDate BETWEEN '2026-01-01'  
-> AND '2026-03-31';
```

5. Index Strategy

Query Requirement	Recommended Index
Search by AccountNumber	B+ Tree
AccountNumber + Date range	Composite B+ Tree
Date range across all accounts	B+ Tree on Date
Exact TransactionID lookup	B+ Tree/Hash

6. Performance

Without an index:

$$O(N)$$

records may need to be scanned.

With a B+ Tree:

$$O(\log BN + K)$$

where K represents the number of matching records.

Conclusion

The primary indexing strategy should be a **composite B+ Tree on (AccountNumber, TransactionDate)**. A separate date index can be added when global date-based queries are frequent. This design efficiently handles both account-based retrieval and date-range queries while remaining scalable for large banking datasets.

3. Healthcare System – PatientID and Doctor Name

Question

A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.

Answer

1. Problem Understanding

A healthcare database may contain millions of patient records.

Example:

- PatientID
- PatientName
- DoctorName
- Department
- Diagnosis
- AppointmentDate

Two important access patterns are:

Search using PatientID

and:

Search using DoctorName

PatientID is normally unique, while many patients can be associated with the same doctor.

Therefore, a single index is insufficient for optimal access.

2. Proposed Multi-Level Indexing

I recommend:

Level 1 – Primary B+ Tree

Create a B+ Tree on:

PatientID

This provides fast direct patient retrieval.

Level 2 – Secondary B+ Tree

Create a secondary B+ Tree on:

DoctorName

This allows the system to retrieve all patients associated with a doctor.

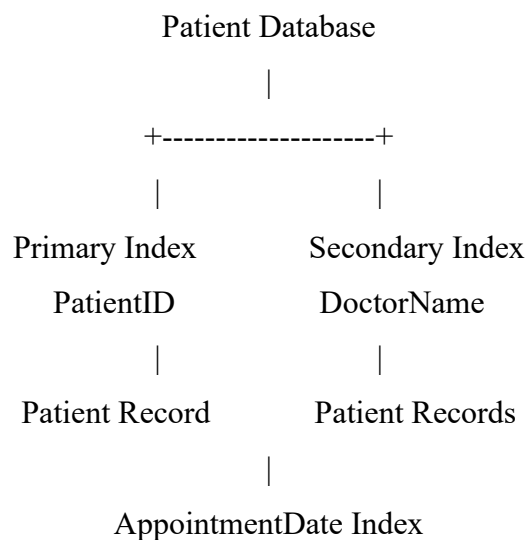
Level 3 – Composite Index

For frequently used doctor-based queries, use:

(DoctorName, AppointmentDate)

This allows efficient retrieval of a doctor's appointments within a date range.

3. Architecture



4. Why B+ Tree?

B+ Trees are suitable because:

- They support equality searches.
- They support range searches.
- They remain balanced.
- They minimize disk accesses.
- Leaf nodes can be linked.

5. Example Queries

PatientID

```
mysql> SELECT *  
-> FROM Patient  
-> WHERE PatientID = 50025;
```

Uses:

PatientID B+ Tree

Doctor

```
mysql> SELECT *  
-> FROM Patient  
-> WHERE DoctorName = 'Dr. Kumar';
```

Uses:

DoctorName secondary index

Doctor + Date

```
mysql> SELECT *  
-> FROM Appointment  
-> WHERE DoctorName = 'Dr. Kumar'  
-> AND AppointmentDate BETWEEN '2026-08-01'  
-> AND '2026-08-31';
```

Uses:

(DoctorName, AppointmentDate)

6. Performance

Without indexing:

$$O(N)$$

With B+ Tree indexing:

$$O(\log BN)$$

for locating the required key, followed by retrieval of matching records.

Conclusion

A multi-level indexing strategy consisting of a **primary B+ Tree on PatientID, secondary B+ Tree on DoctorName, and optional composite index on (DoctorName, AppointmentDate)** provides efficient access for both unique patient retrieval and doctor-based searches.

4. Logistics Company – B-Tree vs B+ Tree

Question

A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.

Answer

1. Problem Understanding

A logistics system continuously performs:

- Shipment insertion.
- Shipment deletion.
- Shipment tracking.
- Shipment searching.
- Date-based shipment retrieval.
- Customer-based retrieval.
- Status-based retrieval.

Therefore, the data structure must efficiently support **dynamic updates while maintaining balanced search performance**.

2. B-Tree

A B-Tree stores keys and associated records in both internal and leaf nodes.

Advantages:

- Balanced structure.
- Efficient insertion.
- Efficient deletion.
- Efficient search.

However, range queries are less efficient compared with B+ Trees because the records are not necessarily concentrated only in linked leaf nodes.

3. B+ Tree

- A B+ Tree stores actual records in leaf nodes.
- Internal nodes contain only separator keys.
- The leaf nodes are linked:
 - Leaf 1 → Leaf 2 → Leaf 3 → Leaf 4
- This makes range queries very efficient.

4. Comparison

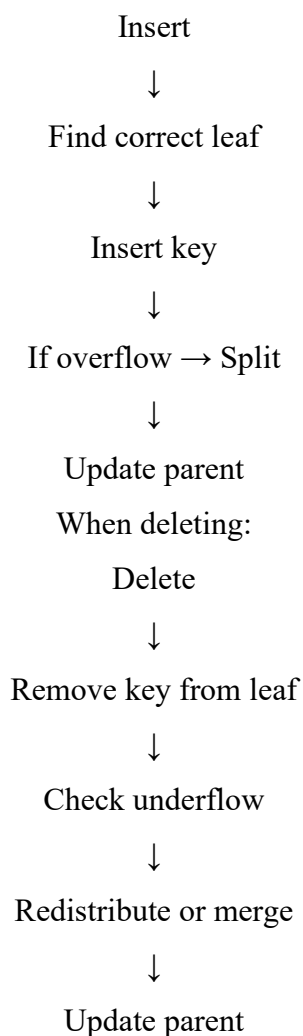
Feature	B-Tree	B+ Tree
Search	Excellent	Excellent
Insert	Excellent	Excellent
Delete	Excellent	Excellent
Range queries	Good	Excellent
Sequential access	Good	Excellent
Leaf linking	Usually absent	Present
Database usage	Less common	Very common

5. Recommendation

- I recommend a **B+ Tree**.
- A logistics system frequently performs range-based queries such as:
 - Find shipments between August 1 and August 15.
- or:
 - Find all shipments from Chennai to Bangalore.
- The linked leaves of a B+ Tree make these operations efficient.

6. Dynamic Updates

When inserting a new shipment:



The tree remains balanced.

Conclusion

Although both B-Trees and B+ Trees support efficient dynamic insertion and deletion, the **B+ Tree is more suitable for the logistics application** because it provides efficient range queries, sequential traversal, high fan-out, and predictable disk-based performance.

5. HR System – 5,000 Employees

Question

Design a hashing scheme for an HR system with 5,000 employees. Evaluate static hashing vs. extendible hashing.

Answer

1. Problem Understanding

The HR system stores employee records such as:

- EmployeeID
- EmployeeName
- Department
- Designation
- Salary

The most common exact lookup is:

EmployeeID

For example:

- Find employee with EmployeeID = 3025

Hashing is highly suitable for this type of equality-based query.

2. Static Hashing

Suppose we choose:

- Number of buckets = 1000

Hash function:

For:

EmployeeID = 3025

we get:

Therefore, employee 3025 is stored in bucket 25.

Collisions can be handled using:

Chaining.

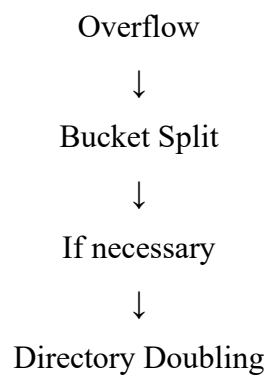
3. Extendible Hashing

Extendible hashing uses:

- Global depth.
- Local depth.
- Directory.
- Dynamic bucket splitting.

When a bucket becomes full, it can split.

If required, the directory is doubled.



4. Comparison

Feature	Static Hashing	Extendible Hashing
Bucket count	Fixed	Dynamic
Implementation	Simple	More complex
Collision handling	Chaining/overflow	Bucket splitting
Growth handling	Poorer	Excellent
Memory overhead	Low	Higher
Suitable for fixed 5,000 records	Excellent	Good
Suitable for future growth	Moderate	Excellent

5. Recommendation

If the HR system is expected to remain close to **5,000 employees**, static hashing with chaining is simpler and cost-effective.

However, real HR systems usually grow due to:

- Recruitment.
- Organizational expansion.
- Mergers.
- Multiple branches.
- Employee history.

Therefore, for a production system with unpredictable growth, I recommend:

Extendible hashing

Because it dynamically adapts to increasing records without requiring complete rebuilding of the hash structure.

Conclusion

For a fixed 5,000-record HR system, static hashing is simpler and cheaper. For a scalable industry system where employee count can grow significantly, **extendible hashing provides better long-term scalability and avoids costly complete rehashing.**

6. Airline Reservation – Flight Number and Date Range

Question

An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.

Answer

1. Problem Understanding

The airline system needs two major access patterns:

Point Query

Find flight with FlightNumber = AI202

Range Query

Find flights between August 1 and August 15.

A single simple index may not optimally support both requirements.

2. Proposed Combined Structure

I recommend:

Index 1 – B+ Tree on FlightNumber

Used for exact flight-number searches.

FlightNumber → Flight Record

Index 2 – B+ Tree on Date

Used for date-range queries.

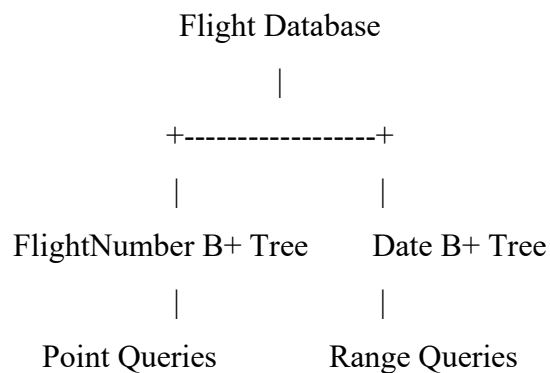
Date → Flight Records

An optional composite index can be created:

(FlightNumber, FlightDate)

if queries commonly combine both fields.

3. Architecture



4. Example

Point query

```
mysql> SELECT *
-> FROM Flight
-> WHERE FlightNumber = 'AI202';
```

Uses:

FlightNumber B+ Tree

Range query

```
mysql> SELECT *
-> FROM Flight
-> WHERE FlightDate BETWEEN '2026-08-01'
-> AND '2026-08-15';
```

Uses:

FlightDate B+ Tree

5. Why B+ Tree?

B+ Tree supports:

- Exact search.
- Range search.
- Ordered traversal.
- Efficient insertion.
- Efficient deletion.

The linked leaf nodes are particularly useful for date-range retrieval.

Conclusion

The recommended solution is a **combined B+ Tree indexing strategy with one index on FlightNumber and another on FlightDate**. This provides fast point queries and efficient date-range queries while maintaining good performance for a dynamic reservation database.

7. University Student Database – Department and CGPA

Question

For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.

Answer

1. Problem Understanding

A university student database may contain:

StudentID

StudentName

Department

Year

CGPA

Typical queries include:

- Find all CSE students.

and:

- Find CSE students with CGPA between 8.0 and 9.5.

The second query combines equality and range conditions.

2. File Organization

I recommend storing student records in a **sorted/organized file using StudentID as the primary key**.

StudentID provides:

- Unique identification.
- Stable primary access.
- Ordered storage.
- Efficient primary indexing.

3. Secondary Index

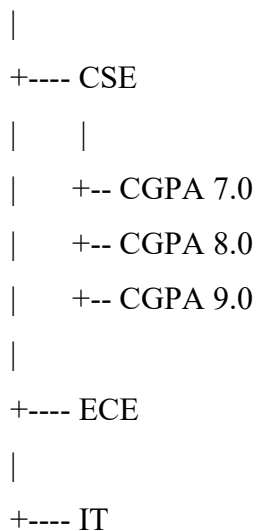
Create a secondary B+ Tree on:

(Department, CGPA)

This is a composite secondary index.

Structure:

Department



4. Query

For:

Department = CSE

CGPA = 8.0 to 9.5

the database:

1. Locates CSE in the index.
2. Locates the beginning of the CGPA range.
3. Traverses the B+ Tree leaves.
4. Retrieves matching student records.

5. Performance

Without secondary indexing:

$O(N)$

With a B+ Tree:

$O(\log BN + K)$

where K is the number of matching students.

6. Why Not Hashing?

Hashing is excellent for exact equality queries:

Department = CSE

but not ideal for:

CGPA BETWEEN 8.0 AND 9.5

because range ordering is not naturally preserved by hashing.

Therefore, B+ Tree indexing is more suitable.

Conclusion

The recommended design is a **StudentID-based primary sorted file with a secondary composite B+ Tree index on (Department, CGPA)**. This provides efficient department filtering and CGPA-range retrieval while maintaining scalable student-record management.

8. Disk Block Utilization – Heap, Sorted and Hashed Files

Question

Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given:

- Records = **100,000**
- Record size = **50 bytes**
- Block size = **4 KB**

Answer

1. Given Data

Number of records:

$$N=100,000$$

Record size:

$$R=50 \text{ bytes}$$

Block size:

$$B=4\text{KB}$$

Assuming:

$$1\text{KB}=1024 \text{ bytes}$$

therefore:

$$4\text{KB}=4096 \text{ bytes}$$

2. Records Per Block

The number of complete records that can fit into one block is:

$$\lfloor 4096/50 \rfloor$$

$$=\lfloor 81.92 \rfloor$$

$$=81 \text{ records}$$

Therefore, each block can store **81 records**.

3. Actual Used Space Per Block

$$81 \times 50 = 4050 \text{ bytes}$$

Unused space:

$$4096 - 4050 = 46 \text{ bytes}$$

Therefore, the unused space per block is:

$$46 \text{ Bytes}$$

4. Number of Blocks

$$\begin{aligned}\text{Blocks} &= \lceil 100000/81 \rceil \\ &= 1235 \text{ blocks}\end{aligned}$$

Therefore, approximately **1,235 blocks** are required.

5. Storage Utilization

Total data stored:

$$100000 \times 50 = 5,000,000 \text{ bytes}$$

Total allocated block space:

$$1235 \times 4096 = 5,058,560 \text{ bytes}$$

Utilization:

$$\begin{aligned}& (5,000,000/5,058,560) \times 100 \\ & \approx 98.84\%\end{aligned}$$

Therefore, the theoretical block utilization is approximately:
98.84%

6. Heap File

A heap file stores records without maintaining a particular ordering.

Advantages

- Simple insertion.
- Good storage utilization.
- No need to maintain sorting.

Disadvantage

Searching may require many block accesses.

7. Sorted File

A sorted file maintains records in a particular order.

Advantages

- Efficient ordered retrieval.
- Binary search can reduce search cost.
- Good for range queries.

Disadvantage

Insertion may require movement or reorganization of records.

8. Hashed File

A hashed file uses a hash function to determine the destination bucket.

For example:

Advantages

- Very efficient equality searches.
- Average constant-time bucket access.

Disadvantages

- Collisions.
- Overflow blocks.
- Poor suitability for range queries.

9. Comparison

Feature	Heap	Sorted	Hashed
Record capacity/block	81	81	81*
Approx. blocks	1235	1235	$\geq 1235^*$
Basic utilization	High	High	Depends on overflow
Insertion	Excellent	Moderate	Excellent
Equality Search	Poor	Good	Excellent
Range Search	Poor	Excellent	Poor
Ordering	No	Yes	No

* Hashed organization can require additional overflow blocks depending on bucket allocation and load factor.

Conclusion

For the given fixed-size records, the theoretical raw block capacity is **81 records per 4 KB block**, requiring approximately **1,235 blocks** and achieving about **98.84% storage utilization** before file-specific metadata, free space, and hashing overflow are considered. The organizations differ primarily in retrieval and update performance rather than the basic record-packing calculation.

9. Social Media Platform – 500 Million Posts

Question

A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.

Answer

1. Problem Understanding

A social media platform with **500 million posts** is a very large-scale database.

Each post may contain:

- PostID
- UserID
- Timestamp
- Hashtag
- Content

Common queries include:

- Find posts created by a particular user.
- Find a user's posts during a particular time period.
- Find posts containing a particular hashtag.
- Find recent posts for a user.

A single index is unlikely to efficiently support all these access patterns.

2. Recommended Composite Index

For user-based timeline queries, create:

- (UserID, Timestamp)

as a composite B+ Tree index.

The ordering should preferably be:

- UserID → Timestamp

This allows the system to locate a particular user first and then efficiently retrieve their posts within a timestamp range.

3. Hashtag Index

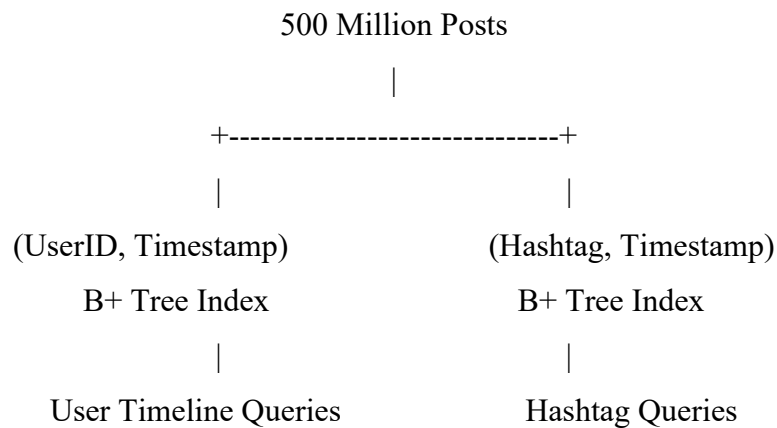
Create a separate index:

- (Hashtag, Timestamp)

This is useful for queries such as:

- Find posts with #AI during the last 24 hours.

4. Proposed Architecture



5. Why Not One Three-Column Index?

A single:

- (UserID, Timestamp, Hashtag)

index may not efficiently support hashtag-first queries because composite indexes are strongly influenced by their **leading column**.

For example:

WHERE UserID = 100

AND Timestamp > ...

works well.

But:

WHERE Hashtag = '#AI'

cannot use the same index as efficiently because Hashtag is not the leading column.

Therefore, separate indexes are justified.

6. Scaling Considerations

500 million posts create substantial index size.

Therefore, an industry system should consider:

- Partitioning by time.
- Partitioning/sharding by user or workload.
- Index compression.
- Index maintenance policies.
- Hot/cold data separation.
- Replication for high availability.

For example:

Recent Posts



Frequently accessed



Fast storage + active indexes

Older Posts



Less frequently accessed



Cold storage/archive

Conclusion

The recommended strategy is to create a **composite B+ Tree index on (UserID, Timestamp)** for user timeline queries and another **composite index on (Hashtag, Timestamp)** for hashtag and time-based retrieval. At 500 million records, partitioning/sharding and index-size management are also essential for scalable production performance.

10. Supermarket Billing System – 1 Million Daily Transactions

Question

Compare the performance of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.

Answer

1. Problem Understanding

A supermarket billing system processes approximately:
transactions per day.

A transaction could contain:

- TransactionID
- CustomerID
- ProductID
- DateTime
- Quantity
- Amount
- PaymentMethod

The system must support:

- Rapid transaction insertion.
- Transaction retrieval.
- Transaction deletion/correction.
- Daily reports.
- TransactionID lookup.
- Time/date-based analysis.

2. Heap File

A heap file stores records in no particular order.

Insert

New transactions can simply be appended.

$O(1)$

approximately, assuming free space is available.

Search

A transaction lookup may require sequential scanning.

$O(N)$

Delete

Requires locating the record first.

$O(N)$

in the basic model.

Best Use

High-volume append operations where searches are infrequent or handled through separate indexes.

3. Sorted File

- Records are maintained in sorted order, for example:
 - TransactionID

Search

Binary search can provide:

Insert

- Maintaining physical order can require record movement.
- Therefore insertion can be expensive.

Delete

- Deletion may also require reorganizing the file.

Best Use

- Applications requiring frequent ordered and range-based retrieval.

4. Hashed File

A hash function maps TransactionID to a bucket.

For example:

$$h(\text{TransactionID}) = \text{TransactionID} \bmod M$$

Search

Average:

$O(1)$

Insert

Average:

$O(1)$

Delete

Average:

$O(1)$

provided collisions and overflow are controlled.

Best Use

Exact transaction lookup.

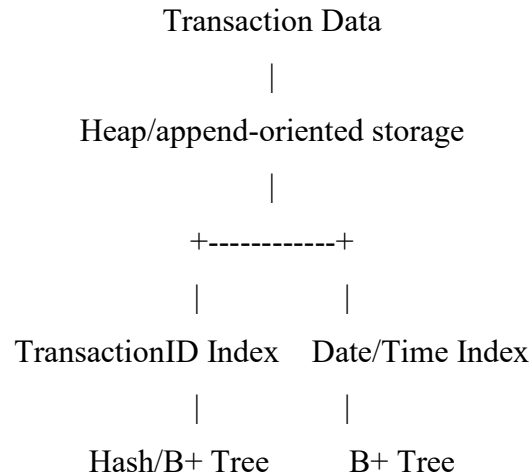
5. Comparison

Operation	Heap File	Sorted File	Hashed File
Insert	$O(1)$	$O(N)$ worst-case	$O(1)$ average
Search	$O(N)$	$O(\log N)$	$O(1)$ average
Delete	$O(N)$	$O(N)$ worst-case	$O(1)$ average
Range Query	Poor	Excellent	Poor
Ordering	No	Yes	No
Collision	No	No	Yes
Maintenance	Low	High	Moderate

6. Recommended Design

For a supermarket billing system, I would **not rely on one file organization for every workload**.

A practical design is:



Why?

Transaction insertion is extremely frequent.

Therefore, append-oriented storage is attractive.

For exact transaction lookup:

- TransactionID → Hash Index

For reports such as:

- Transactions between 10:00 AM and 11:00 AM

or:

- Transactions between August 1 and August 25

a B+ Tree on date/time is more appropriate.

7. Performance Analysis

For 1 million transactions:

Heap

- Searching all records can approach
- record comparisons in the worst case.

Sorted

- Binary search requires
- comparisons for an in-memory conceptual model.
- Actual database performance is determined by **block/disk I/O**, not only CPU comparisons.

Hashed

- Average lookup approach
- bucket access when the hash function distributes records well and overflow is controlled.

8. Final Recommendation

- For a supermarket billing system, **hashed organization is the best choice for exact TransactionID-based retrieval**, while a **B+ Tree is preferable for date/time range reporting**.
- If the requirement is primarily high-speed transaction insertion, a heap/append-oriented file with appropriate indexes can also be highly effective.
- Therefore, the most practical industry solution is a **hybrid design** rather than forcing every operation into one file organization.

Conclusion

Heap files provide the simplest and fastest insertion, sorted files provide efficient ordered and range retrieval, and hashed files provide excellent average-case exact-key retrieval. For a supermarket handling **1 million transactions daily**, a hybrid architecture using **append-oriented storage, hashing for exact TransactionID lookups, and B+ Tree indexing for date/time range queries** provides the best balance of insertion speed, retrieval efficiency, scalability, and reporting capability.